O Level :Web Designing & Publishing

Infomax Computer Academy

Chapter 6 : JAVA SCRIPT

Objective:

- Understand the JavaScript syntax and structures.
- Learn, how JavaScript programs are used in a web pages including the use of event handlers and the Document Object Model (DOM)
- Insert JavaScript into a web page using various approaches including inline code, internal scripts and external JavaScript files
- Write and use simple JavaScript functions and event handlers that reflect common applications of JavaScript

What is JavaScript?

- JavaScript is client Side Scripting language that runs in client system.
- JavaScript is an interpreted computer programming language.
- JavaScript is a programming language designed for Web pages.
- JavaScript enhances Web pages with dynamic and interactive features.
- JavaScript was developed in 1995 by Brendan Eich, at Netscape, and first released with Netscape 2 early in 1996.
- It was initially called LiveScript, but was renamed JavaScript to capitalize the popularity of Sun Microsystems's Java language.

JavaScript Terminology.

- JavaScript programming uses specialized terminology.
- Understanding JavaScript terms is fundamental to understanding the script.
  - Objects
  - Properties
  - Methods
  - Events
  - Functions
  - Values
  - Variables,
  - Expressions
  - Operators

Objects

- Objects refers to windows, documents, images, tables, forms, buttons or links, etc.
- Objects should be named.
- Objects have properties that act as modifiers.

Properties

- Properties are object attributes.
- Object properties are defined by using the object's name, a period, and the property name.
  - e.g., background color is expressed by: **`document.bgcolor`** .
  - **`document`** is the object.
  - **`bgcolor`** is the property.

Events

- Events associate an object with an action.
  - e.g., the **`OnMouseover`** event handler action can change an image.
  - e.g., the **`onSubmit`** event handler sends a form.
- User actions trigger events.

Mouse Events

Event	Description
onclick	Script to be run on a mouse click
ondblclick	Script to be run on a mouse double-click
onmousedown	Script to be run when mouse button is pressed
onmousemove	Script to be run when mouse pointer moves
onmouseout	Script to be run when mouse pointer moves out of an element
onmouseover	Script to be run when mouse pointer moves over an element
onmouseup	Script to be run when mouse button is released

Keyboard Events

Event	Description
onkeydown	Script to be run when a key is pressed
onkeypress	Script to be run when a key is pressed and released
onkeyup	Script to be run when a key is released

Form Events

Event	Description
onblur	Script to be run when an element loses focus
onchange	Script to be run when an element changes
onfocus	Script to be run when an element gets focus
onreset	Script to be run when a form is reset
onselect	Script to be run when an element is selected
onsubmit	Script to be run when a form is submitted

<body> and <frameset> Events

Event	Description
onload	Script to be run when a document load
onunload	Script to be run when a document unload

Image Events

Event	Description
onabort	Script to be run when loading of an image is interrupted

Methods

- Methods are actions applied to particular objects. Methods are what objects can do.
  - e.g., document.write(”Hello World") **(”Hello World")**
  - **`document`** is the object.
  - **`write`** is the method.

Methods Example

- alert(): to show any message in alert box

  Sy: alert(“text”) or alert(variable name)

  Ex. Alert(“plz enter your Name”)

- confirm(): to get confirmation from user in form of ok and cancel

  Sy: var x= confirm(“message”)

  Ex. var name = confirm("Are you sure?")

- Prompt() : to accept some data from user

  Sy: var x=prompt(“message”, “ ”) or var x=prompt(“message”, value)

  Ex. var name= Prompt(“Enter your Name”, “ “)

Open/Close Method

- open() : Open New Window

  Sy. var var_name

  var_name=window.open("file_name",
  "width=value, height=value");

  Ex. var newWin

  newWin= window.open("test.html");

- close() : to close current Window

  Sy. Window.close();

Method Example

- window.print() : to open a print dialog box
- document.write() : to write some text/html in webpage
- console.log() : to write some data on console
- eval() : to evaluate any mathematical expression
- start() : to start the scroll text in marquee which stopped by calling stop()
- stop() : to stop the scroll text in marquee .

Functions

- Functions are named statements that performs some tasks.
  - e.g.,

```
function doWhatever ()
{
    statement here
}
```

- The curly braces contain the statements of the function.
- JavaScript has built-in functions, and you can write your own.

Values

- Values are bits of information.
- Values types and some examples include:
  - Number
    - Integer : 1,-5,8
    - Real Number : 1.6,- 2.7, 3.0, etc.
  - Text:
    - Single Character: ‘a’, ‘%’ ,’8’ characters enclosed in single quotes.
    - String (set of char) : “infomax”, “Abc@123”
  - Boolean: true or false.
  - Object : date, time ,file etc.

Variables

- In JavaScript, you can declare variables using the var, let, or const keywords.
- The var keyword is used to declare a variable. Variables declared with var have function scope or are globally scoped if declared outside a function.
  - Example : var age = 25;
- The let keyword is used to declare a block-scoped variable, meaning the variable is only available within the block (e.g., inside a loop, if statement, or function) where it is defined.
  - Example : let name = "Jane";
- The const keyword is used to declare a constant variable that cannot be reassigned after its initial value is set. Like let, it is also block-scoped.
    - Example: const PI = 3.14159;

Expressions

- Expressions are commands that assign values to variables.
- Expressions always use an assignment operator, such as the equals sign.
  - e.g., `var month = May;` is an expression.
- Expressions end with a semicolon.

Operators

- Operators are used to handle variables.
- Types of operators with examples:
  - Arithmetic operators
  - Assignment Operator
  - Relational Operator
  - Logical operators
  - Bitwise Operator
  - Increment Decrement Operator
  - Conditional Operator

Arithmetic Operators

- An arithmetic operator performs mathematical operations such as addition, subtraction and multiplication on numerical values (constants and variables).

Operator	Description	Example (int a=10,b=7;)
+	Addition	a+b =17
–	Subtraction	a-b = 3
*	Multiplication	a*b = 70
/	Division	a/b = 1
%	Modulus (Remainder)	a%b = 3

Relational Operators

- A relational operator checks the relationship between two operands.
- It returns Boolean value i.e. true or false
- Relational operators are used in decision making and loops.

Operator	Description	Example (int a=5,b=10)	Result
>	greater than	a > b	false
<	less than	a < b	true
>=	greater than or equal to	a >= b	false
<=	less than or equal to	a <= b	true
==	equal to	a == b	false
!=	not equal to	a != b	true
===	is a strict equality operator. It checks if two values are equal in both value and type		

Logical Operators

- Logical operators are used to combine two or more condition for building complex expression.
- It also return a Boolean result i.e. true or false.

Operator	Name	Description
&&	AND Operator	Checks whether the conditions preceding and succeeding it is true or not.
||	OR Operator	Checks whether either of the conditions preceding and succeeding it is true or not.
!	NOT Operator	Just negates the logic of the condition succeeding it to check for its validity.

Assignment Operator

- In Java, the assignment operator (=) is used to assign a value to a variable.
  - a=5;
  - x=y;
- Chained Assignments: You can chain assignments to assign the same value to multiple variables in a single statement.
  - b=b=c=5;
- Compound Assignment Operators: it combine an operation and assignment in a single statement.

+= -= *= /= %=

Increment/Decrement Operator

- Increment operators are used to increase the value of the variable by one and decrement operators are used to decrease the value of the variable by one.

Type of Increment Operator:

- Pre increment/decrement Operator (++a, --a)
- Post increment/decrement Operator (a++, a--)

- In pre increment / decrement first increment/decrement the value of variable and then used inside the expression (initialize into another variable)
- In post-increment / decrement first value of variable is used in the expression (initialize into another variable) and then increment / decrement the value of variable.

Conditional Operator

The conditional operator (? :) is a ternary operator. This operator needs 3 operands to perform the operation.

Syntax:

variable = <test expression> ? <expression 1> : <expression 2>

The conditional operator works as follows:

- Test Expression is Condition
- Expression 1 is Statement Followed if Condition is True
- Expression 2 is Statement Followed if Condition is False

Precedence and Associativity of operators

Precedence:

- it determines the order in which operators are evaluated in expressions.
- Operators with higher precedence are evaluated first, followed by operators with lower precedence.

Associativity:

- If operators have the same precedence, they are evaluated from left to right (left-associativity) or from right to left (right-associativity), depending on the specific operators.

Operator	Description	Associativity
( ) [ ] . -> ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Methods of Using JavaScript.

- JavaScript object attributes can be placed in HTML element tags.

  e.g., `<body onlclick="alert('WELCOME')">`

- JavaScript can be embedded in HTML documents - in the **<head>**, in the **<body>**, or in both.

- JavaScripts can reside in a separate page. The extension of javaScript file must be .js ex: data.js

1. Embedding JavaScript in HTML

- When specifying a script only the tags **<script>** and **</script>** are essential, but complete specification is recommended:

```
<script language="javascript”>
      <!-- Begin hiding
          document.write(”JavaScript Code");
      // End hiding script-->
</script>
```

2. Using Separate JavaScript Files

- Linking can be advantageous if many pages use the same script.
- Use the source element to link to the script file.

```
<script src="myjavascript.js”
language="JavaScript1.2”>
</script>
```

Accessing Form Element

- document.form_name.field_name.property_name;
- Ex:
- Var x=document.frm1.t1.value;

Accessing Form element

- Accessing form elements in JavaScript can be done using various methods, typically by interacting with the DOM.
- Accessing Form Elements
  - By ID: var a = document.getElementById('elementId');
  - By Name: var a = document.getElementsByName('elementName');
  - By Tag Name: var a = document.getElementsByTagName('tagName');
  - By Class Name: var a = document.getElementsByClassName('className');

Applying CSS using Javascript

- Document.getElementById(“Id_name”).style.property_name=value;
- Example
- Document.getElementById(“box”).style.backgdound=“red”;

get or set the HTML content

- Getting the HTML Content:

```
var content = document.getElementById("myDiv").innerHTML;
console.log(content);
```

- Setting the HTML Content:

```
document.getElementById("myDiv").innerHTML = "<p>New content goes here!</p>";
```

Appending a Text Node to a Div Element

- Syntax:

  parentElement.append(child1, child2, ..., childN);

- Example:

```
// Select the parent element
var div = document.getElementById("myDiv");
// Create a new text node
var newText = document.createTextNode(" This is the appended text.");
// Append the text node to the div
div.append(newText);
```

Appending Multiple Elements

```
<ul id="myList">
        <li>Item 1</li>
        <li>Item 2</li>
 </ul>
```

```
<script>
    // Select the parent element
    var ul = document.getElementById("myList");

    // Create new elements
    var newListItem1 = document.createElement("li");
    var newListItem2 = document.createElement("li");

    // Add content to the new elements
    newListItem1.textContent = "Item 3";
    newListItem2.textContent = "Item 4";

    // Append the new list items to the unordered list
    ul.append(newListItem1, newListItem2);
</script>
```

remove an element from the DOM

- Syntax: element.remove()
- Example:

```
function removeElement() {
    // Select the element to remove
    var element = document.getElementById("myDiv");
    // Remove the element from the DOM
    element.remove();
}
```

Removing Multiple Elements

```
<ul id="myList">
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
    <li>Item 4</li>
</ul>
```

```
function removeItems() {
    // Select all list items
    var items = document.querySelectorAll("#myList li");

    // Loop through the NodeList and remove each item
    items.forEach(function(item) {
        item.remove();
    });
}
```


Infomax

Conditional/Selection statement

- **Decision making statement** is depending on the condition block need to be executed or not which is decided by condition. In java programming language there are three types of conditional statement:

- If statement
- If else statement
  - Nested if else statement
  - Else if ladder statement
- Switch case statement

if Statement

- This is the simplest form of branching statement. When we have only one condition to evaluate, we should use this statement. It performs a course of action if the condition evaluates to true, otherwise, it ignores the action.
- **Syntax:**

```
If(conditional expression)
{
    Statement 1
    Statement 2
    ……..
    Statement n
}
```

if-else statement

The if else control structure is used where either of two action are to be performed depending upon the result of a conditional expression. It contains two block of statement. If the conditional expression evaluates to true, the statement(s) in if block are executed and if the result is false, then statement(s) in the else block get executed.

Syntax:

```
if(conditional expression)
{
        statement 1
        statement 2
        ……….
}
else
{
        statement 1
        statement 2
        ……….
}
```

IF ELSE IF LADDER

- The if-else-if ladder statement executes one condition from multiple statements. The execution starts from top and checked for each if condition. The statement of if block will be executed which evaluates to be true. If none of the if condition evaluates to be true then the last else block is evaluated.

```
if(condition_expression_One)
{
   statement1;
}
else if (condition_expression_Two)
{
   statement2;
}
else if (condition_expression_Three)

{
   statement3;
}
else
{
   statement4;
}
```

Nested IF ELSE

- A nested if is a statement that has another if or else in its body.
- Hear, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
If(condition)
{
    If(condition)
    {
        Statement
    }
}
else
{
    Statement
}
```

Switch Statement

A switch statement is used when there is requirement to check multiple condition in a program. It provides an easy way to branch to different parts of the code in program based on the value of an expression. It is a substitute for the large series of else if statements.

Syntax:

```
switch(expression)
{
        case constant:
                statement;
        break;
        case constant:
                statement;
        break;
        ..
        ..
        default:
                statement;
}
```

Looping statement

A for loop is used to repeat a block of statement enclosed in curly braces. It contains three optional expressions enclosed in parentheses and separated by semicolons, followed by statements executed in the loop.

- For loop
- While loop
- Do while loop

For Loop

for loop is a statement which allows code to be repeatedly executed. For loop contains 3 parts Initialization, Condition and Increment or Decrements.

Syntax:

```
for(initial value ;condition; increment/decrement)
{
        statement 1
        statement 1
        ……….
        statement n
}
```

While loop

The while loop is entry controlled loop. The while loop keeps on executing the block of statement till the specified test condition evaluates to true. When the test condition becomes false, the program control is transferred out of the loop and passes to the statement after the body of the loop.

Syntax:

```
while(condition)
{
        Statement 1;
        Statement 2;
        ……..
        Statement n;
}
```

do-while Loop

The do while loop is an **exit controlled loop**, since the body of loop is executed first and then the test condition is evaluated. If it evaluates to false, then the loop is terminated, otherwise it is repeated. This means do while loop always executes **at least once**, even if the condition is evaluated to false.

Syntax:

```
do
{
        statement 1;
        statement 1;
        ……….
        statement 1;
}while(condition);
```

Text Function in JavaScript

- Length : length is not function it is property to get length of string
- charAt(index): Returns the character at the specified index.
- indexOf(searchValue, fromIndex): Returns the index of the first occurrence of a specified value.
- lastIndexOf(searchValue, fromIndex): Returns the index of the last occurrence of a specified value.
- startsWith(searchString, position): Checks if a string starts with another string.
- endsWith(searchString, length): Checks if a string ends with another string.
- concat(...strings): Combines two or more strings.

Text Function in JavaScript

- replace(searchValue, newValue): Replaces occurrences of a specified value.
- substring(start, end): Extracts characters from a string between two specified indices.
- substr(start, length): Extracts a substring from a string.
- toLowerCase(): Converts a string to lowercase.
- toUpperCase(): Converts a string to uppercase.
- trim(): Removes whitespace from both ends of a string.
- trimStart(): Removes whitespace from the beginning of a string.
- trimEnd(): Removes whitespace from the end of a string.

Math function in JavaScript

- Math.PI: Pi, approximately 3.14159 (Math Constants not function)
- Math.abs(x): Returns the absolute value of x.
- Math.ceil(x): Returns the smallest integer greater than or equal to x.
- Math.floor(x): Returns the largest integer less than or equal to x.
- Math.round(x): Returns the value of x rounded to the nearest integer.
- Math.trunc(x): Returns the integer part of x, removing any fractional digits.
- Math.sign(x): Returns the sign of the number: 1 if positive, -1 if negative, and 0 if zero.

Math function in JavaScript

- Math.sqrt(x): Returns the square root of x.
- Math.cbrt(x): Returns the cube root of x.
- Math.pow(base, exponent): Returns base raised to the power of exponent.
- Math.log(x): Returns the natural logarithm (base E) of x.
- Math.log10(x): Returns the base-10 logarithm of x.
- Math.exp(x): Returns E raised to the power of x.
- Math.sin(x): Returns the sine of x (in radians).
- Math.cos(x): Returns the cosine of x (in radians).

Math function in JavaScript

- Math.tan(x): Returns the tangent of x (in radians).
- Math.max(...values): Returns the largest of zero or more numbers.
- Math.min(...values): Returns the smallest of zero or more numbers.
- Math.random(): Returns a pseudo-random number between 0 and 1.

Date and Time Function

- new Date(): Creates a new date object with the current date and time.
- new Date(dateString): Creates a new date object from a date string.
- Date.now(): Returns the number of milliseconds elapsed since January 1, 1970, 00:00:00 UTC.
- date.getDate(): Returns the day of the month (1-31).
- date.getDay(): Returns the day of the week (0-6).
- date.getFullYear(): Returns the year (4 digits).
- date.getHours(): Returns the hours (0-23).

Date and Time Function

- date.getMilliseconds(): Returns the milliseconds (0-999).
- date.getMinutes(): Returns the minutes (0-59).
- date.getMonth(): Returns the month (0-11).
- date.getSeconds(): Returns the seconds (0-59).
- date.getTime(): Returns the number of milliseconds since January 1, 1970.

THANK YOU

Address: G. R. Complex Preetam Nagar, Dhoomanganj
Prayagraj Uttar Pradesh 211011

Mobile No.: 9598948810,8874588766,9532878816
Website : www.infomax.org.in

Infomax Computer Academy